Book: A Tour to C++

Introduction

Start

- Core Langauage Features: Char and Int
- Standard Library Components: Vector and Map
- C++ Statically Typed Language: Menas type of every entity(object, value, name and expression) must be known to the compiler at its point of use. the type of an object determine the set of operations applicable to it
- Type of Function: Return Type and sequence of argument type
  - `Ex- double get(const vector<double>&vec, int index) // type: double(const vector<double&>, int)`
  - Function Can be member of class , then name of calss also part of the function type, `type: char&Tring::(int)`
- Function overloading

```
void print(int,double);
void print(double, int);
print(0,0); // compiler erro : ambiguous
```

Types, Variables and Arithmetic

- A ***declaration*** is a statement that introduces an entity into the program. It specifies the type for the entity
  - A ***type*** defines a set of possile values and a set of operation (for an object)
  - An ***object*** is some memory that holds a value of some type
  - A ***value*** is a set of bits interpreted according to a type
  - A ***variable*** is a name object
- We can use ` this in long literals ex 3.14159`263535
- Char variable is of the natural size to hold a character on given machine (typically 8-bit)
- The conversion used in expression are called ***the usuall arithmetic conversion*** aim to ensure that expression are computed at the highest precision of its operands.
- **Order of evaluation** The order of evaluation is left-to right for `x.y, x->y, x(y), x[y], x<<y, x>>y, x&&y, and x||y`. For assignments `(e.g., x+=y)`, the order is right-to-left. For historical reasons realated to optimization, the order of evaluation of other expressions `(e.g., f(x)+g(y))` and of function arguments `(e.g., h(f(x),g(y)))` is unfortunately unspecified.

  > **Note:** Evaluation and Associtaivty both are different

Initialization

- ex

```
double d1 = 2.3;
double d2 {2.3};
double d3 = {2.3}; // the = is optional with {...}
complex<double>z2 {d1,d2}
complex<double>z3 = {d1, d2} // the = is optional with {...}
```

- {} this saves you from conversion that losse information
  - ex, int i1 = 7.8 // i becomes 7 (surprise?)
    - unfortunately, conveersion that loose information, ***narrowing conversions***, such as double to int and int to char are allowed and implitcly applied.
  - ex, int i2{7.8} // error: floating-point to integer conversion
- Don"t need to state its type explicitly when it can be deduced from the initializer
  - auto b = true; // a bool
  - auto ch = 'x'; // a char
  - use in generic programming where the long type names and the exact type of an object can be hard for programmer to know

> ***Parentheses ()*** call the most specific matching constructor and allow implicit conversions, but they can be mistaken for function declarations. ***Braces {}*** provide "Uniform Initialization" that prevents data loss and solves ambiguity, but they will always prioritize an initializer_list constructor if one is available.

Scope and Initialization

- Scope
  - Local Scope - Declared in a function or lambda is called a local name.
  - Class Scope- Class Member Name, outside of any function, lambda or enum class
  - Namespace Scope - outside of any function, class, or enum class.
- For a namespace object the point of destruction is the end of the program, For a member the point of destruction is determine by the point of destruction oth object of which it is a member

Constants

- Two notions of immutability
  - ***const***: "I promise not to change this value". The compiler enforces the promise made by const. The value of a *const* can be calculated at run time.

    ```
    const int runtimeValue = std::rand();  // OK, value not known at
    compile time
    runtimeValue = 5;  // Error: cannot assign to const variable
    ```

    > **Note:** If a runtime-initialized const variable is placed in read-only memory, it triggers a memory protection fault (segmentation fault) when written to at runtime.

- ***constexpr***: to be evaluated at compile time, to allow placement of data in read only memory, the value of a contexpr must be calculated by the compiler

```
constexpr int compileTimeValue = 42;          //OK
constexpr int bad = std::rand();              //Error: rand() is
not constexpr
```

  - Funtion to be usable in a *constant exppression*, must define contexpr

```
constexpr double square(double x){return x*x;}
constexpr double max1 = 1.4*square(17);// ok, evaluated at
compiletime
constexpr double max2 = 1.4*square(var);// error: var is not
a constant expression
const double max3 =1.4*square(var);// ok, may be evaluated
at runtime
```

  - contexpr function can be used for non constant argument, so we can use the same funcntion for constexpr and for variables, without defining two.

> **Note:** c++20, consteval introduce, if we want only be evaluated at compile time then define with **consteval**

  - contexpr or consteval functions cannot modify non-local variables, but it can have loop[s and use its own local variable

Pointers, Array and Refernces

- Pointers and Array
  - In declaration
    - `[]` means: **array of**
    - `*` menas: **pointer to**
    - `&` means **refernce to**
  - In expression
    - `*` means: **contents of**
    - `&` means: **address of**
- C++ rang-for statement, for loops that traverse a sequence in the simplest way

```
void print()
{
    int v[]={0,1,2,3,4,5};
    for(auto x:v) // for each x in v
     cout << x << '\n';

    for(auto x:{10,21,32,35})
        cout << x << '\n';
```

```
    for(auto& x:v) // add 1 to each x in v
    {
        ++x;
        cout << x << '\n';
    }
}
```

- Declarator operators

```
T a[n] // T[n]: a is an array of n Ts
T* p   // T*: p is a pointer to T
T& r   // T&: r is a reference to T
Tf(A)  // T(A), is a funtion argument of type A and return type T
```

- Null Pointer

```
Link<Record>*Ist = nullptr; // pointer to a link to a Record
int x = nullptr; //error:nullptr id a ponter not an integer
```

- reference cannot be made to refer to a different object after its initialization.
- There is no “null reference.” A reference must refer to a valid object (and implementations assume that it does). There are obscure and clever ways to violate that rule; don’t do that.

Tests

- Like a for-statement, an if-statement can introduce a variable and test it.

```
if(auto n = v.size(); n!=0) // the integer n defined for use within
the if-statement and intialized with v.size
{
    // ...we get here if n!=0
}

// simplified version
 if(auto n = v.size())
{
    // ...we get here if n!=0
}
```

Mapping to hardware

- Assignments

- An assignment of built-in type is a simple machine copy operation, 'int x =2; int y=3', both are independent object
- Assignement to a refernce, does not change what the reference refers to but assigns to the referenced object
- Assignment replaces the value of an already initialized object.

```
int x = 2;
int y = 3;
int& r =x;
int& r2 = y;
r = r2; // read through r2, write through r: x becomes 3
```

- To access the value pointed to by a pointer, you use *; that is automatically (implicitly) done for a refernce.

- Initilization
  - Initialization differs from assignment. In general, for an assignment to work correctly, the assigned-to object must have a value. On the other hand, the task of initialization is to make an uninitialized piece of memory into a valid object
  - You can use = to initialize a reference but please don't let that confuse you. `int& r = x; // bind r to x (r refers to x)`This is still initialization and binds r to x, rather than any form of value copy.

> **Note:** The basic semantics of argument passing and function value return are that of initialization.

- Ex
  - Initialization

```
// Creating a variable and giving it a value at the same time.
int x = 10;           // Initialization (built-in type)
std::string s = "hi"; // Initialization (calls constructor)
```

  - Assignment

```
// Changing the value of an already initialized variable.
x = 20;               // Assignment (built-in type: overwrite)
s = "hello";          // Assignment (calls assignment operator)
```

User Defined Types

Introduction

- The C++ abstraction mechanism are primarily designed to let programmers design and imp[lement their own type.
- ***User-defined types (classes and enumeration)*** are less error prone and easy to use than built-in types
- structure

```
struct Vector
{
    int sz;
    double *elem;
};


void vector(Vector& v, int s)
{
    v.elem = new double[s]; // allocate an array of s doubles
    v.sz =  s;
}

void f(Vector v, Vector& rv, Vector* pv)
{
    int i1 = v.sz;    // access through name
    int i2 = rv.sz;  // access through reference
    int i3 = pv->sz; //access through the pointer
}
```

Classes

- A tighter connection between the representation and the operation is needed for a user-defined type to have all the properties expected of a **real-type**. To do that we have to distinguish between the interface to a type (to be used by all) and its implementation (which has access to the otherwise inaccssible data). The language mechnaism of for that is called a class.
- The interface defined by public member of the class and private member only accessible only through that interface.

> '***Note:*** Vector(int s):elem{new double[s]},sz{s}{}' member **initializer list**, can only be used in a constructor.

```
class Vector {
public:
    Vector(int s):elem{new double[s]},sz{s}{} //contruct a vector
    double& operator[](int i){return elem[i];} //element access:
subscripting
    int size(){return sz;}
private:
    doouble* elem; // pointer the elemennts
    int sz // the number of elements
}
```

```
double read_and_sum(int s)
{
    Vector v(6);
    for(int i=0; i<v.size(); ++i)
        cin >>v[i];

    double sum = 0;
    for(int i=0; i<v.size(); ++i)
        sum +=v[i];

    return sum;
}
```

Unions

- Occupies only as much space as its largest element

```
struct Entry{
    str name;
    Type t;
    Node* p; // use p if t==ptr
    int i;   // use i if t==num
}

union Value{
    Node* p;
    int i;
};

struct Entry{
    str name;
    Type t;
    Value v;
}
```

> **Note:** shoul not use naked union, alway use with type or call it tagged union

- **varaint**, standard library type can eliminate most direct use of unions, `variant<Node*, int>, holds_alternative<int>(pe->v),get<int>`

```
struct Entry{
    string name;
    variant<Node*, int> v;
}

void f(Entry *pe)
{
    if(holds_alternative<int>(pe->v))
```

```
        cout << get<int>(pe->v);
}
```

Enumerations

- Ex

```
enum class Color {red, blue, green};
enum class traffic_light{green, yellow, red};
Color x = red; //error: which red?
Color y = Traffic_light::red; //error:that red is not a Color
Color z = Color::red; //ok

int i = Color:: red //error: Color::red is not an int
Color c = 2 //initialization error:2 is not a color
```

> ***Note:*** enum - allow implicit conversion, enum class not allow implicit conversion - only explixit conversion allowed, means class after enum specifies that an enumeration is *strongly typed*. Also using enum class also specifies thier scope, menas being separate types

```
Color x = Color{5}; // ok, but verbose
Color y{6}; //also ok
```

- By default **enum class** has only assignment, intialization and comparison defined. However an enumeartion is user defined type, so we can define operator for it:

```
Traaffic_light& operator++(Traffic_light& t)
{
    switch(t){
        case Traffic_loght::green: return t=Traffic_light::yellow;
        case Traffic_light::yellow return t=Traffic_light::red;
        case Traffic_light::red return t=Traffic_light::green;
    }
}

Traffic_light next = ++light; // next becomes Traffic_light::green
```

- dont want to explicitly qualify enmuerator names and want enumerator values to be ints (without need for an expliti conversion) remove the **class** from enum.

```
enum Color {red, green, blue};
int col =  green; // col get 1
```

Advice

- Organize related data into structures
- Prefer enum class over plain enum
- Avoid naked unions
- define contructors to gurantee and simplyfy the intitialization of class
- Define operation on enumerations for safe and simple use

Modularity

Introduction

- A ***declaration*** specifies all that's need to use a function or a type. `double sqrt(double);`

Separate compilation

- A .cpp file that is compiled by itself is called a **translation unit**.

```
@startuml

rectangle "Vector.h Interface" as VectorInterface
rectangle "Vector.cpp \n #include "Vector.h"" as VectorImplementation
rectangle "user.cpp \n #include "Vector.h"" as UserCode

UserCode -up-> VectorInterface
VectorImplementation -up-> VectorInterface

@enduml
```

Modules (C++20)

- Ex

```
export modules vector; // defining the module called "vector"

export class Vector {
public:
    Vector (int s);
    double& operator[](int i);
    int size();
private:
    double* elem;
    int sz;
};

Vector::Vector(int s):elem{new double[s]}, sz{s}{}

double& Vector::operator[](int i)
{
```

```
    return elem[i];
}

int Vector::size()
{
    return sz;
}

export bool operator==(const Vector& v1, const Vector& v2)
{
    if(v1.size()!=v2.size())
    {
        return false;
    }

    for(int i = 0; i<v1.size(); ++i)
    {
        if(v1[i]!=v2[i])
        {
            return false;
        }
    }
    return true;
}
```

- This defines a module called vector, which export the class vector, all is member functions, and the non-member function defining operator ==.

```
// file user.cpp

import Vector; // get Vector's interface
include <cmath> // get the standard-library math function interface
including sqrt()

double sqrt_sum(Vector& v)
{
    double sum = 0;
    for (int i=0; i!=v.size(); ++i)
        sum+std::sqrt(v[i]);
    return sum;
}
```

- #include has many problems as follows
    - Compilation time: will be processed by the compiler in every trnsalation unit
    - Order dependencies: If we #include header1.h before header2.h the declarations and macros in header1.h might affect the meaning of the code in header2.h.
    - Transitivity: All code that is needed to express a declaration in a header file must be present in that header file. This leads to massive code bloat as header

- module benfits
    - A module is compiled once only
    - Two modules can be imported in either order without changing their meaning.
    - If you import or #include something into a module, users of your module do not implicitly gain access to (and are not bothered by) that: import is not transitive.
    - The effects on maintainability and compile-time performance can be spectacular.

Namespaces

- Ex

```
void my_code(vector<int>&x, vector<int>&y)
{
    using std::swap;  // use the std library swap
    //...
    swap(x,y);  // std::swap
    other::swap(x,y); // some other swap
}
```

- A **using**-declaration makes a name from a namespace usable as if itwas declared in the scope in which it appears: `using std::swap`
- To gain access to all names in the std library namespace, we can use a **using**-directive: `using namespace std`

Error Handling

Exceptions

- What ought to be done when we try to access an element that is out of range for the vector example we did?
    - **Vector::operato** can detect an attempted out of range access and throw an **out_ofrange** exception.

```
double& Vector::operator[]()
{
    if(i<0||i>=size())
    {
        throw out_of_range{"Vector::operator[]"};
    }
    return elem[i];
}
```

- The **throw** trnasfer control to a handler for exception of type **out_of_rnage** in some function that directly or indirectly called **Vector::operator**. To do that, the implementation will unwind the function call stack as needed to get back to the context of that caller, functions as needed to get

back to caller that has expressed interest in handling that kind of exception, invoking destructors along the way as needed.

```
void f(Vector&v)
{
    //....
    try{ // exceptions here are handled by the handler defined below
        v[v.size()] = 7; // try to access beyond the end of v
    }
    ctach(out_of_range& err) { // oops: out_of_range error, caught by
reference and what() func to print the error
        // .. handle range error
        cerr << err.what()<<\"\n";
    }
}
```

- The main technique for making error handling simple and systematic (RAII).

> **RAII (Resource Acquisition Is Initialization)**: The basic idea behind is for a constructor to acquire all resources necessary for a class to operarte and have the destructor relase all resources, thus making resource release guranteed and implicit.

- A functions that should never throw an exception can be declared **noexcept**

```
void user(int sz) noexcept
{
    Vector v(sz);
    iota(&v[0], &v[sz],1);
}
```

  - If all good intent and planning fails, so that `user()` still throws, `std::terminate()` is called to immediately terminate the program.

Invariants

- Use of exception to check a ***precondition***, like checking function argument and refusing to act.
- example if we say "elem points to an array of sz doubles" but we only said that in a comment. Such statement of what is asssumed to be true for a class is called a **class invariant**, or simply a **invariant**.
- It is the job of a contructor to establish the invariant for its class(so that the memeber functions can rely on it) and for the member function to make sure that the invariant holds when they exit.
- **Invariant**: A rule that an object must always follow, defining its valid state. It's the class's internal promise to itself.
  - Example: For a BankAccount object, the invariant is "the balance is always non-negative." Every method must ensure this rule is never broken.

- **Precondition:** A rule that a function's caller must follow for the function to work correctly. It's a promise from the user to the function.
    - Example: A divide(a, b) function has the precondition that b is not zero. The function trusts this and can proceed with the division without checking for a zero denominator.
- `Vecotor v(-27);`, this is likely to cause chaos.
- appropriate definition:

```
Vector::Vector(int s)
{
    if (s<0)
        throw length_error{"Vector Constructor: negative size"};
    elem = new double[s];
    sz = s;
}
```

- if a **new** can't find memory to allocate, it trhows a **std::bad_alloc**.

```
void test()
{
    try{
        Vector v(-27);
    }
    catch(std::length_error& err){
        // handle negative size
        cerr << "test failed:length error\n";
        throw; //rethrow
    }
    catch (std::bad_alloc& err){
        // handle memory exhaustation
        std::terminate();
    }
}
```

- "rethrowing" an exception is the process of catching an exception, performing some actions (like logging or cleanup), and then throwing the same exception again to be caught by a different catch block higher up the call stack.
- The Two Core Ideas
    - **Don't use try-catch blocks for simple cleanup**: In C++, you often have to handle things like closing files, releasing memory, or unlocking a mutex. Manual try-catch blocks are a clumsy way to do this because you have to write the cleanup code in multiple places—once for a successful path and again for a failed (exception) path.
    - **Use RAII instead**: RAII is a C++ technique where you wrap a resource (like a file handle or a pointer) inside a class. This class's constructor acquires the resource, and its destructor

automatically releases it. The C++ language guarantees that the destructor will be called when the object goes out of scope, whether the function ends normally or because an exception was thrown.

```
#include <fstream>
#include <iostream>

void process_file_raii(const std::string& filename) {
    // The resource is acquired in the constructor
    std::fstream file(filename);

    if (!file) {
        throw std::runtime_error("Could not open file.");
    }

    // No try-catch block needed for cleanup!
    read_data_from_stream(file);

    // When the function returns or an exception is thrown,
    // 'file' goes out of scope, and its destructor is called
automatically,
    // which closes the file handle.
}
```

- C++ constructors establish a class's internal rules (invariants) by correctly acquiring resources, while destructors maintain those rules by guaranteeing resources are released, even during errors. This core mechanism, known as RAII, is central to robust C++ resource management.

Error Handling Alternatives

- We return an error indicator(an "error code") when:
    - A failure is normal and expected
    - An immediate caller can reasonably be expected to handle the failure
- We trhow an exception when
    - An error is so rare
    - An error cannot be handled by an immediate caller. Instead the error has to percolate back to an ultimate caller. For ex, it is infeasible to have every function in an application reliably handle every allocation failure or network outage.

      > **Note:** The C++ runtime system unwinds the stack, skipping over all the functions in the call chain until it finds a catch block that is prepared to handle that type of exception.

    - Not suitable return path, ex constructor
    - The function that found the error was callback
    - An error implies that some "undo action" is needed
    - The error has to be transmitted up a call chain to an ultimate caller

- We terminate when
  - An error cannot recover
  - non-trival error is detected

> **Note:** One way to ensure termination is to add noexcept to a function so that a throw from anywhere in the function's implementation will turn into a terminate(). Ex. If a function marked noexcept calls another function that does throw an exception, or if it throws an exception itself, the C++ runtime environment will immediately call std::terminate(). **Note:** If an exception escapes a noexcept function during stack unwinding, the runtime stops unwinding and immediately calls std::terminate(), skipping destruction of any remaining stack objects.

Assertions

- The standard library offers the debug macro, assert(), to assert that a condition must hold at run time. `assert(p!=nullptr); // p must not be the nullptr`

```
enum class Error_action { ignore, throwing, terminating, logging };
// error-handling alternatives

constexpr Error_action default_Error_action = Error_action::throwing;
// a default

enum class Error_code { range_error, length_error };
// individual errors

string error_code_name[] { "range error", "length error" };
// names of individual errors

template<Error_action action = default_Error_action, class C>
constexpr void expect(C cond, Error_code x) // take "action" if the
expected condition "cond" doesn't hold
{
if constexpr (action == Error_action::logging)
    if (!cond()) std::cerr << "expect() failure: " << int(x) << '' <<
error_code_name[int(x)] << '\n';
if constexpr (action == Error_action::throwing)
    if (!cond()) throw x;
if constexpr (action == Error_action::terminating)
    if (!cond()) terminate();
// or no action
}

double& Vector::operator[](int i)
{
    expect([i,this] { return 0<=i && i<size(); },
Error_code::range_error);
    return elem[i];
}
```

Static Assertions

- We can also perform simple checks on most properties that are known at compile time and report failure to meet our expectation as compiler error message.

```
static_assert(4<=sizeof(int),"Integers are too small");

constexpr double C = 2999792.458; // km/s

void f(double speed)
{
    constexpr double local_max = 160.0/(60*60);        // 160 km/h ==
160.0/(60*60) km/s

    static_assert(speed<C,"can't go that fast");       // error: speed
must be a constant
    static_assert(local_max<C,"can't go that fast");   // OK

    // ...
}
```

- One important use of static_assert is to make assertions about types used as parameters in generic programming

```
template <typename T>
void only_accept_integers(T value) {
    // This static_assert checks if T is an integer type at compile
time.
    static_assert(std::is_integral_v<T>, "Error: This function only
works with integer types.");

    std::cout << "The value is: " << value << std::endl;
}

int main() {
    only_accept_integers(10);        // OK: T is int
    only_accept_integers(50L);       // OK: T is long
    // only_accept_integers(3.14);    // COMPILE-TIME ERROR: T is
double
    // only_accept_integers("hello"); // COMPILE-TIME ERROR: T is
const char*

    return 0;
}
```

Argument passing

- Default function argument

```
void print(int value, int base =10);
print(x,16);
print(x,60);
print(x);
```

Function Argument and Return Values

- Passing information to and from functions
  - Is an object copied or shared?
  - If an object is shared, is it mutable?
  - Is an object moved, leaving an "empty object" behind?

Value return

- Ex

```
Matrix operator+(const Matrix& x, const Matrix& y)
{
    Matrix res;
    //...
    return res;
}

Matrix m1, m2;
//..
Matrix m3=m1+m2; // no copy
```

- A matrix may be very large and expensive to copy even on mordern hardware. So we dont copy, we give matrix a move contructor and very cheaply move the Matrix out of operator+()

```
Matrix * add(-cont Matrix& x, const Matrix& y)
{
    Matrix* p = new Matrix;
    //..
    return p;
}

Matrix m1, m2;
//..
Matrix*m3 = add(m1,m2); //just copy a pointer
//..
delete m3; //easily forgotten
```

> **Note:**returning a large object using pointer to its common in older code, and major source of hard to find errors. Dont write such code.

- The return type of a function can be deduced from its value, ex`auto mul(int i, double d) {return i*d;} // here "auto means deduced the rerun type"`

Structural Binding

- A function can return only a single value, but that value can be a class object with many members. This allows us to effieciently return many values.

```
struct Entry {
    string name;
    int value;
}

Entry read_entry(istream& is)
{
    string s;
    int i;
    is >> s >> i;
    return {s,i}; // Imp- is used to contruct the Entry return type
}

auto e = read_entry(cin);

cout << "{"<< e.name<<","<<e.value<<"}\n";
// another method - this strunctural binding
auto [n,v] = read_entry(is);
cout << "{"<< n<<","<<v<<"}\n";
```

- Here, return {s,i} is used to contruct the Entry return type
- The `auto[n,v]` declares two local variables n and v with their types deduced from read_entry()'s return type. This mechanism for giving local names to members of a class object is called **structural binding**.
- When structured binding is used for a class with no private data. There muste be the same number of names defined for the binding as there are nonstatic data members of the class.

```
complex<double>z={1,2};
auto[re, im] = z+2 // re =3; im =2
```

Advice

- Don’t apply noexcept thoughtlessly
- Use an assertion mechanism to provide a single point of control of the meaning of failure
- Let a constructor establish an invariant, and throw if it cannot

Classes

Introduction

Concrete Class

- its representation is part of its definition, such as **vector** class, to increase flexibility a concrete tpe can keep major parts of its representation on the free store (dynamic memory, heap) and access them through the part stored in the class object itself. That's the way *vector* and *string* are immplemented. resources handles with interfaces.
- class contains operation requiring access to the representation

> **Function defined in a class are inlined by default**

- const specifier on the functions indicate that these function do not modify the object for which they are called.
- const member function can be invoked for both const and non const objects, but for non const fuction can only be invoked for non const objects.

```
double real() const {return re;}

complex z = {1,0};
const complex cz {3,4};
z = cz; // OK:assigning to a non const variable
cz = z; // error:complex::operator=() is a non consta member function
double x = z.real(); // ok: complex::real() is a const member function
```

- operations does not require direct access to the representation they can be defined separately from the class definition

```
complex operator+(complex a, complex b){return a+=b;}
```

- The compiler converts operators operation into appropriate function calls.

```
c!=b means operator!=(c,b);
1/a means operator/(complex{1},a)
```

- operator chaining , Assignment (=) T&, Stream (<<, >>) ostream& / istream&, Prefix ++ T&, Postfix ++ T, Arithmetic (+, -), T (usually)

```
ostream& operator<<(ostream& os, const MyClass& obj) {
os << obj.value;
return os;   // 🔑 key
}
```

> In C++, operator overloading is allowed only if at least one operand is a user-defined type (class, struct, or enum). (c1+=c2)+=c3 if return as value then Operation is performed on temp object, so the original object is unchanged. user define operators use cautiously, syntax is fixed by language so you can't define a unary /.

> Plain **delete** deletes and individual object, **delete[]** deletes an array.

- RAII - Resource Acquisition Is Iintialization - avoid naked new and delete and burried them iinside well - behaved abstractions
- A ***handle to a data model*** is an indirect reference that lets you access or control the data without owning or exposing it directly.

```
Model m{10};
Model& handle = m;   // handle
handle.value = 20;
```

> Return by reference because array indexing yields an lvalue (the actual element). Returning by value would create a temporary, so `const double& r = v[0];` would bind to a temporary copy.

intializing containers

- best way to intialize
  - ***Initializer-list constructor***: Initialize with list of elements
  - ***push_back***: Add a new element at the end of the sequence

```
class Vector {
public:
    Vector(std:initializer_list<double>); // initialize with a list of doubles
    //...
    void push_back(double); // add element at end, increase the size by one
}
```

- ***std::intializer_list*** define a intializer-list constructor is a std lib type known to the compiler: when we use a {}-list, such as {1,2,3,4} compiler will create an object of type initializer_list to give to the program, ex `Vector v = {1,2,3,4}`
- A static_cast does not check the value it is converting; the programmer is trusted to use it correctly.
- Other casts are reinterpret_cast and bit_cast (§16.7) for treating an object as simply a sequence of bytes and const_cast for “casting away const.”

Abstract Types

- an abstract type is a type that completely insulates a user from its implementation. to do that decouple the interface from its representation and give up the genuine local variables
- an abstract class can have methods with definitions

```
class Container {
public:
    virtual double& operator[](int) = 0;     // pure virtual function
    virtual int size() const = 0;            // const member function
(§5.2.1)
    virtual ~Container() {}                  // destructor (§5.2.2)
};
```

- there cannot be object of abstract class measn it cannot be instantiate
- a class with a pure virtual function is called an ***abstract class***, an class provide the interface to other class is often called ***polymorphic type***
- abstract class does not have constructor as it does not have data to initialize
- that is common for abstract classes because they tend to be manipulated through references or pointers, and someone destroying a Container through a pointer has no idea what resources are owned by its implementation
- A virtual destructor is essential for an abstract class because an object of a derived class is usually manipulated through the interface provided by its abstract base class. In particular, it may be deleted through a pointer to a base class. Then, the virtual function call mechanism ensures that the proper destructor is called. That destructor then implicitly invokes the destructors of its bases and members.
- use of **override** is optional, but being explicit allows the catch mistake like typo in function name

> ***NOTE:*** members objects are constructed in the order they are declared in the class definition not the order they apperar in the constructor member initializer list

Virtual Functions

- The compiler to convert the name of a virtual function into an index into a table of pointers to functions.
- Each class with virtual functions has its own vtbl identifying its virtual functions.
- Its space overhead is one pointer in each object of a class with virtual functions plus one vtbl for each such class. - vptr location, vtable index

Benefits from Hierarchies

- Interface Inheritance
- Implementation Inheritance

> Objects are constructed "bottom up" (base first) by constructors and destroyed "top down" (derived first) by destructors.

- if we want use a function which only provided by a particular derived class then we use ***dynamic_cast***

```
Shape* ps{read_shape(cin)};
if(Smiley* p = dynamic_cast<Smiley*>(ps)) { //... does ps point to a
smiley
    // ....a smiley; use it
} else {
    //... not a smiley
}
```

> dynamic_cast use case - if we pass the object to system where it uses the base class interaface then when it return we need to recover to original derived class

Avoiding Resources Leaks

- use a standard-library unique_ptr rather than a “naked pointer” when deletion is required:

```
class Smiley : public Circle {
    // ...
private:
    vector<unique_ptr<Shape>> eyes; // usually two eyes
    unique_ptr<Shape> mouth;
};

unique_ptr<Shape> read_shape(istream& is) // read shape descriptions
from input stream is
{
    // ... read shape header from is and find its Kind k ...

    switch (k) {
    case Kind::circle:
        // ... read circle data {Point,int} into p and r ...
        return unique_ptr<Shape>{new Circle{p,r}};          // §15.2.1
    // ...
}

void user()
{
    vector<unique_ptr<Shape>> v;

    while (cin)
        v.push_back(read_shape(cin));

    draw_all(v);                            // call draw() for each
element
    rotate_all(v,45);                       // call rotate(45) for
each element
} // all Shapes implicitly destroyed
```

Advice

- Prefer concrete classes over class hierarchies for performance-critical component
- Make a function a member only if it needs direct access to the representation of a class
- Use nonmember functions for symmetric operators
- Declare a member function that does not modify the state of its object const
- If a class is a container, give it an initializer-list constructor
- A class with a virtual function should have a virtual destructor
- Use override to make overriding explicit in large class hierarchies
- When designing a class hierarchy, distinguish between implementation inheritance and interface inheritance
- Use dynamic_cast where class hierarchy navigation is unavoidable
- Use dynamic_cast to a pointer type when failure to find the required class is considered a valid alternative

```
vector v;                           // object goes out of scope → destructor
~vector();

vector a = b;                       // create new object from existing → copy constructor
vector(const vector&);

v = b;                              // overwrite existing object → copy assignment
vector& operator=(const vector&);

vector x = make_vector();           // create from temporary → move constructor
vector(vector&&);

v = std::move(x);                   // overwrite using temporary → move assignment
vector& operator=(vector&&);
```

- **lvalue**: An expression that refers to a named object with identity and stable storage (e.g., `v`, `*ptr`); you can take its address.
- **prvalue**: An expression that produces a value without naming an object (e.g., `10`, `3+4`, `vector(10)`); since C++17 it doesn't create a temporary—used directly for initialization (guaranteed elision).
- **xvalue**: An "expiring" object with identity that is about to be moved from (e.g., `std::move(v)`).

> ***prvalue object example*** `vector v = vector(3);` What happens (C++17+): vector(3) is a prvalue, No temporary vector is created, v is constructed directly, Equivalent to:`vector v(3);`

```
vector v = vector(3);   // prvalue → direct construction
vector u = make();      // prvalue return → guaranteed elision
```

> reference to x via pointer

```
    int x = 10;
    int* a = &x;  // a points to x
    int& p = *a; // p is now a reference to x

    void move_all(vector<unique_ptr<shape>>& v, Point x ){
        // std::unique_ptr cannot be copied to ensure unique ownership
of a resource
        // Use std::move to transfer ownership instead of copying
        for(auto &p:v){
            p->move(x);
        }
    }

    unique_ptr<shape> ps{read_shape(kind::smiley,{2,4}, 4, 0)};
    //Like a for-statement, an if-statement can introduce a variable
and test it.
    // ps.get();              // get raw pointer
    // p.get() returns the raw pointer managed by the unique_ptr.
    // Ownership remains with the unique_ptr, so you don't manually
delete raw.
    if (smiley* p = dynamic_cast<smiley*>(ps.get()); p != nullptr) {
        p->wink_eye();
    }
```

Essential Operations

program need to implement

- copy and move all the scenarios including conversion
- implement example for implicit and explicit conversion with () = {}
- move constructor assignment with prints, copy and move for all the scenarios with print like in function call and check what compiler call move or copy

Essential Operations

```
X(Sometype);            // "ordinary constructor": create an object
X();                    // default constructor
X(const X&);            // copy constructor
```

```
    X(X&&);                  // move constructor
    X& operator=(const X&); // copy assignment: clean up target and copy
    X& operator=(X&&);      // move assignment: clean up target and move
    ~X();                    // destructor: clean up
```

- there are five places where object can be copied and moved:
  - as the source of an assignment

    ```
    A a1(10);
    A a2(20);
    a2 = a1;        // copy assignment operator called
    ```

  - as an object initializer

    ```
    // 2. As an object initializer → creating a new object from
    another object
    A a1(10);
    A a2 = a1;      // copy constructor called
    // note:: A a = 10;     → Copy initialization → 10 converted to
    A(10) → constructor runs
    ```

  - as a function argument

    ```
    // 3. As a function argument → object passed by value to a
    function
    void func(A x) {}
    A a1(10);
    func(a1);       // copy constructor (or move) used
    ```

  - as a fuction return value

    ```
    // 4. As a function return value → returning object by value from
    function
    A make() {
        A temp(10);
        return temp;    // copy/move may happen (often optimized by
    RVO)
    }
    A a1 = make();
    ```

  - as a execption

```
// 5. As an exception → object copied when thrown/caught by value
try {
    A a1(10);
    throw a1;       // copy constructor used
}
catch(A e) { }      // another copy when catching by value
```

> a copy or move constructor invocation is often optimized away by constructing the object used to initialize right in the target object. compiler will typically construct the X from make() directly in x; thus eliminating ***("eliding")*** a copy

```
X make(Sometype);
X x = make(value);
```

- initialization of named objects and of objects on the free store, constructors are used to initialize temporary objects and to implement explicit

```
        class A{ public: A(int x){} };
        A a(10);          // named object initialization
        A* p = new A(5); // object created on free store (heap)
        A b = A(20);      // temporary object conceptually created
(usually copy elision)
        A c = 30;         // implicit conversion: int → A using
constructor
        A d = A(30);      // explicit conversion: int → A, here
compiler will do the rvo
```

- Except for the "ordinary constructor," these special member functions will be generated by the compiler as needed

```
    class Y {
    public:
        Y(Sometype);
        Y(const Y&) = default;    // I really do want the default copy
constructor
        Y(Y&&) = default;         // and the default move constructor
        // ...
    };
```

> *rule of zero* either defined all or none for move and copy operators and constructor

- to complement =default, we have =delete to indicat that an operation is not to be generated

> A base class object should not be copied because derived objects may contain additional data not present in the base class. Copying a derived object into a base object copies only the base

> part (object slicing), causing the derived part to be lost.

```
    void copy(Shape& s1, const Shape& s2)
```

Conversions

- `complex z1 = 3.14; // z1 becomes {3.14,0.0}` this is implicit conversion

```
    class Vector {
    public:
        explicit Vector(int s);    // no implicit conversion from int
to Vector
        // ...
    };
    Vector v1(7);   // OK: v1 has 7 elements
    Vector v2 = 7;  // error: no implicit conversion from int to
Vector
```

Member Initializers

- The default value is used whenever a constructor doesn't provide a value. This simplifies code and helps us to avoid accidentally leaving a member uninitialized.

```
class complex {
    double re = 0;
    double im = 0; // representation: two doubles with default value
0.0
```

Copy and Move

- When we design a class, we must always consider if and how an object might be copied. For simple concrete types, memberwise copy is often exactly the right semantics for copy. For some sophisticated concrete types, such as Vector, memberwise copy is not the right semantics for copy; for abstract types it almost never is.

- rvalue and lvalue

```
    x = y;        // OK (x is lvalue)
    x + y = 20;   // ERROR (x+y is rvalue)

    // x + y produces a temporary result, so it is an rvalue.
```

> The **&&** means "rvalue reference" and is a reference to which we can bind an rvalue , int&& a = 10; a binds to that temporary Lifetime of temporary extends to lifetime of a , a → 10. So an

> rvalue is – to a first approximation – a value that you can't assign to, such as an integer returned by a function call.Thus, an rvalue reference is a reference to something that nobody else can assign to, so we can safely "steal" its value.

- RVO: Compiler eliminates copy/move when returning a temporary object.

```
    Vector create() {
        return Vector(100);   // RVO
    }
```

- NRVO: NRVO: Compiler may eliminate copy/move when returning a named local object, but it is not guaranteed by the C++ standard.

```
    Vector create() {
        Vector temp(100);
        return temp;   // NRVO (optional)
    }
    // NRVO cannot apply because two different objects may be
returned.
    Vector create(bool flag) {
        Vector a(100);
        Vector b(200);

        if(flag)
            return a;
        else
            return b;
    }
```

> RVO cannot happen in assignment because the object already exists and RVO only works during object construction. `A a = f(); // initialization`

```
A a;
a = f(); // move constructor work here
```

> After a move, the moved-from object should be in a state that allows a destructor to be run. Typically, we also allow assignment to a moved-from object. that is why we assigned null ptr if destructor call then is should not free the moved object memory

> NOTE: Imp Example below

```
    z = x;             // we get a copy (x might be used later in f())
    y = std::move(x);  // we get a move (move assignment)
    // ... better not use x here ...
```

```
//....
void func(A x);
A a;
func(a);        // copy
func(A(10));    // move
```

- The standard-library function **move()** doesn't actually move anything. Instead, it returns a reference to its argument from which we may move – an rvalue reference; it is a kind of cast